

AutoGen

Multi-Agent Conversation Framework

Introduction

AutoGen is an open-source multi-agent framework originally developed by Microsoft for building applications where multiple AI agents collaborate by exchanging messages, similar to how people communicate in a chat. Instead of a single LLM call producing one answer, AutoGen lets you define several `ConversableAgent` instances — each with its own role, instructions, and optionally a different LLM — and have them talk to each other, negotiate, critique, and reach a final outcome.

AutoGen's defining idea is the conversation as the unit of orchestration: agents send and receive messages just like humans in a group chat, and the framework manages turn-taking, reply limits, and (optionally) human participation in the loop. It supports everything from simple two-agent dialogues to GroupChat sessions with many specialist agents and an LLM-driven manager that decides who speaks next — making it well suited for simulations, negotiations, multi-step approvals, and collaborative problem solving.

Components

Component	Description
<code>ConversableAgent</code>	AutoGen's core agent class; can act with or without an LLM (<code>llm_config=False</code> disables it).
<code>llm_config</code> / <code>config_list</code>	Connects an agent to an LLM backend via an OpenAI-compatible API (e.g., local Ollama).
<code>system_message</code>	Defines an agent's persona, goals, and behavioural constraints.
<code>human_input_mode</code>	Controls human involvement: <code>NEVER</code> (fully autonomous), <code>ALWAYS</code> (always ask a person), <code>TERMINATE</code> (ask only to end).
<code>send()</code> / <code>generate_reply()</code>	<code>send()</code> delivers a message directly; <code>generate_reply()</code> gets one LLM response for a message list.
<code>initiate_chat()</code>	Starts and drives a full back-and-forth conversation between two agents (or an agent and a manager).
<code>GroupChat</code>	Holds multiple agents and a shared message list for multi-party conversations.
<code>GroupChatManager</code>	An LLM-backed orchestrator that selects the next speaker in a <code>GroupChat</code> .
<code>speaker_selection_method</code>	How the next speaker is chosen: <code>round_robin</code> , <code>auto</code> (LLM-decided), <code>random</code> , or <code>manual</code> .
<code>max_turns</code> / <code>max_round</code> / <code>max_consecutive_auto_reply</code>	Safety limits that cap how long a conversation or loop can run.

Syntax

```
# 1. Configure the LLM (OpenAI-compatible endpoint, e.g. local Ollama)
config_list = [{"model": "qwen2.5:3b", "base_url": "http://localhost:11434/v1", "api_key": "ollama"}]
llm_config = {"config_list": config_list, "temperature": 0}

# 2. Create an Agent
from autogen import ConversableAgent
agent = ConversableAgent(
    name="<AgentName>",
    system_message="<role, goal, and instructions for this agent>",
    llm_config=llm_config,          # or False for a no-LLM agent
    human_input_mode="NEVER",      # NEVER | ALWAYS | TERMINATE
    max_consecutive_auto_reply=4
)

# 3a. Two-agent conversation
chat_result = agent_a.initiate_chat(agent_b, message="<opening message>", max_turns=5)

# 3b. Multi-agent GroupChat
from autogen import GroupChat, GroupChatManager
group_chat = GroupChat(agents=[a, b, c], messages=[], max_round=8,
                        speaker_selection_method="round_robin") # or "auto"
manager = GroupChatManager(groupchat=group_chat, llm_config=llm_config)
agent_a.initiate_chat(manager, message="<opening message>")

# 3c. Single reply without a full chat loop
reply = agent.generate_reply(messages=[{"role": "user", "content": "<prompt>"}])
```

Learning Outcomes

- Create ConversableAgents with and without an LLM, and exchange messages directly via send().
- Configure a local LLM (Qwen 2.5 via Ollama) using AutoGen's OpenAI-compatible config_list.
- Run two-agent conversations with initiate_chat() and interpret the returned chat history.
- Build multi-agent GroupChat systems with round-robin and LLM-driven (auto) speaker selection.
- Combine deterministic Python tool functions with LLM agents in a hybrid pipeline.
- Implement human-in-the-loop approval steps using human_input_mode and generate_reply().

Software Requirements

Requirement	Detail
Python Version	3.10 or higher
Core Package	pyautogen==0.3.2
Other Libraries	rich==13.9.4 (console output in Labs 4 and 5)
LLM Backend	Ollama running locally, serving qwen2.5:3b at http://localhost:11434/v1

Requirement	Detail
API Key	None required for Ollama — a placeholder string ("ollama") is used
Environment	Any local Python environment, terminal, or Jupyter/VS Code setup
Install Command	<code>pip install -r requirements.txt (pyautogen==0.3.2, rich==13.9.4)</code>

Lab Programs

Lab 1: Basic AutoGen Agents (No LLM)

Source Code

```
# =====
# HANDS-ON 1 -- Basic AutoGen Agents (No LLM)
# =====

from autogen import ConversableAgent

# -- STEP 1: Create Two Agents with NO LLM -----

teacher = ConversableAgent(
    name="TeacherAgent",
    llm_config=False,
    human_input_mode="NEVER"
)

student = ConversableAgent(
    name="StudentAgent",
    llm_config=False,
    human_input_mode="NEVER"
)

print(f"Agent 1 created: {teacher.name}")
print(f"Agent 2 created: {student.name}")

# -- STEP 2: Send Messages Between Agents -----

print("\n--- Teacher sends a question ---")
teacher.send(
    message="What is Python?",
    recipient=student,
    request_reply=False
)

print("--- Student sends an answer ---")
student.send(
    message="Python is a high-level programming language used for AI, web, and data science.",
    recipient=teacher,
    request_reply=False
)

print("\nMessages exchanged!")

# -- STEP 3: Read Message History -----
# chat_messages = {partner_agent: [{"role": ..., "content": ...}]}

print("\n=== TeacherAgent Message History ===")
```

```

for msg in teacher.chat_messages.get(student, []):
    print(f" [{msg['role'].upper()}] {msg['content']}")

print("\n=== StudentAgent Message History ===")
for msg in student.chat_messages.get(teacher, []):
    print(f" [{msg['role'].upper()}] {msg['content']}")

# -- STEP 4: Three-Agent Classroom -----

print("\n\n=== THREE-AGENT CLASSROOM ===\n")

teacher2 = ConversableAgent(name="Teacher", llm_config=False, human_input_mode="NEVER")
student_a = ConversableAgent(name="StudentA", llm_config=False, human_input_mode="NEVER")
student_b = ConversableAgent(name="StudentB", llm_config=False, human_input_mode="NEVER")

question = "What is the difference between AI and Machine Learning?"

teacher2.send(message=question, recipient=student_a, request_reply=False)
teacher2.send(message=question, recipient=student_b, request_reply=False)

student_a.send(
    message="AI is the broad concept of machines being smart. ML is a subset where machines learn from data.",
    recipient=teacher2, request_reply=False
)
student_b.send(
    message="AI makes machines intelligent. ML lets machines improve through experience without being explicitly programmed.",
    recipient=teacher2, request_reply=False
)

print(f"Teacher asked: {question}")
print("\nAnswers received by Teacher:")
for partner in [student_a, student_b]:
    for msg in teacher2.chat_messages.get(partner, []):
        if msg["role"] == "user":
            print(f" {partner.name}: {msg['content']}")

# -- STEP 5: HR Department Example -----

print("\n\n=== HR DEPARTMENT EXAMPLE ===\n")

hr = ConversableAgent(name="HR", llm_config=False, human_input_mode="NEVER")
developer = ConversableAgent(name="Developer", llm_config=False, human_input_mode="NEVER")
designer = ConversableAgent(name="Designer", llm_config=False, human_input_mode="NEVER")

hr.send(message="Please submit your weekly report by 5 PM today.",
        recipient=developer, request_reply=False)
hr.send(message="Please submit your weekly report by 5 PM today.",
        recipient=designer, request_reply=False)

developer.send(message="Report submitted: Completed authentication module, fixed 3 API bugs.",
               recipient=hr, request_reply=False)
designer.send(message="Report submitted: Completed landing page redesign, delivered 5 new UI components.",
              recipient=hr, request_reply=False)

print("HR broadcast the weekly report request.")
print("\nHR received:")
for partner in [developer, designer]:
    for msg in hr.chat_messages.get(partner, []):
        if msg["role"] == "user":
            print(f" {partner.name}: {msg['content']}")

print("\n=== Hands-on 1 Complete ===")
print("Next step: Hands-on 2 -- Replace llm_config=False with Qwen 2.5 via Ollama")

```

Sample Input / Output

```

teacher.send("What is Python?", recipient=student)
student.send("Python is a high-level language...", recipient=teacher)

--- Output ---
Agent 1 created: TeacherAgent
Agent 2 created: StudentAgent

--- Teacher sends a question ---
--- Student sends an answer ---
Messages exchanged!

=== TeacherAgent Message History ===
[ASSISTANT] What is Python?
[USER] Python is a high-level programming language used for AI, web, and data science.

```

Key Concepts Covered

Concept	Description
ConversableAgent	AutoGen's base agent class; works with or without an LLM (llm_config=False disables it).
send()	Directly delivers a message from one agent to another without invoking an LLM.
chat_messages	A dict of {partner_agent: [messages]} recording full conversation history per pair.

Lab 2: Retail Chat using Local LLM (Qwen 2.5 via Ollama)

Source Code

```
# =====
# HANDS-ON 2 -- Retail Chat using Local LLM (Qwen 2.5 via Ollama)
# =====

from autogen import ConversableAgent

print("\n=== HANDS-ON 2: RETAIL CHAT WITH LOCAL LLM ===\n")

# -- STEP 1: LLM Configuration -----
# AutoGen uses an OpenAI-compatible API format.
# Ollama exposes this at http://localhost:11434/v1

config_list = [
    {
        "model": "qwen2.5:3b",
        "base_url": "http://localhost:11434/v1",
        "api_key": "ollama",
    }
]

llm_config = {
    "config_list": config_list,
    "temperature": 0, # 0 = deterministic responses
}

print("LLM Config:")
print(f"  Model : {config_list[0]['model']}")
print(f"  Endpoint: {config_list[0]['base_url']}")

# -- STEP 2: Create Agents with system_message -----

customer_agent = ConversableAgent(
    name="CustomerAgent",
    system_message=(
        "You are a college student in India looking for a smartphone.\n"
        "Your budget is Rs 20,000.\n"
        "You want: good camera for Instagram, long battery life (all-day use), "
        "and smooth performance for college apps.\n"
        "Ask at least 2 specific questions before deciding.\n"
        "When you are satisfied, say PURCHASE CONFIRMED on its own line."
    ),
    llm_config=llm_config,
    human_input_mode="NEVER",
    max_consecutive_auto_reply=4
    # max_consecutive_auto_reply: how many times this agent replies in a row
    # without a break. Safety guard against infinite looping.
)

retail_agent = ConversableAgent(
    name="RetailAdvisor",
    system_message=(
        "You are an expert smartphone retail advisor in India.\n"
        "You know every phone under Rs 20,000: Redmi Note series, Poco X series, "
        "Samsung M series, Realme Narzo, iQOO Z series.\n"
        "For every recommendation include: model name, price in Rs, "
        "camera megapixels, battery mAh, and one key strength.\n"
        "Be concise. When customer says PURCHASE CONFIRMED, "
        "reply with a one-line congratulation and purchase summary."
    ),
    llm_config=llm_config,
    human_input_mode="NEVER",
    max_consecutive_auto_reply=4
)
```

```

print(f"\nAgent 1: {customer_agent.name} -- budget Rs 20,000")
print(f"Agent 2: {retail_agent.name} -- smartphone expert")

# -- STEP 3: Start the Conversation with initiate_chat() -----

print("\n--- Starting retail conversation ---\n")

chat_result = customer_agent.initiate_chat(
    retail_agent,
    message="Hi! I need a smartphone under Rs 20,000 for college. "
           "I want a great camera and all-day battery. What do you suggest?",
    max_turns=5
)

# -- STEP 4: Conversation Summary -----

print("\n\n=== CONVERSATION SUMMARY ===")
print(f"Total messages exchanged: {len(chat_result.chat_history)}")
print("\nFull transcript:")
for i, msg in enumerate(chat_result.chat_history):
    name = msg.get("name", msg.get("role", "?"))
    content = msg.get("content", "")
    short = content[:200] + ("..." if len(content) > 200 else "")
    print(f"\n[{i+1}] {name}: \n {short}")

# -- STEP 5: Different Scenario -- Laptop Purchase -----

print("\n\n=== SCENARIO 2: LAPTOP PURCHASE ===\n")

student_buyer = ConversableAgent(
    name="VITStudent",
    system_message=(
        "You are a VIT University student buying a laptop for Python and Java coding "
        "and watching video lectures. Budget: Rs 55,000. "
        "You need at least 8GB RAM and 8+ hour battery. "
        "Ask 2 questions before deciding. Say LAPTOP CONFIRMED when ready to buy."
    ),
    llm_config=llm_config,
    human_input_mode="NEVER",
    max_consecutive_auto_reply=3
)

laptop_advisor = ConversableAgent(
    name="LaptopAdvisor",
    system_message=(
        "You are a laptop retail expert in India. "
        "Recommend models under Rs 55,000 for coding and study. "
        "Always mention: model name, price, RAM, storage, battery life, processor. "
        "Narrow to ONE best recommendation after the student asks questions. "
        "When student says LAPTOP CONFIRMED, give a short purchase summary."
    ),
    llm_config=llm_config,
    human_input_mode="NEVER",
    max_consecutive_auto_reply=3
)

laptop_chat = student_buyer.initiate_chat(
    laptop_advisor,
    message="Hi, I need a laptop under Rs 55,000 for college. "
           "I will use it for Python coding and online classes.",
    max_turns=5
)

```

Sample Input / Output

```

"Hi! I need a smartphone under Rs 20,000 for college.
I want a great camera and all-day battery. What do you suggest?"

--- Output ---
CustomerAgent: Could you tell me the camera megapixels and
battery capacity of your top 2 options?

RetailAdvisor: Redmi Note 13 - Rs 18,999 - 108MP camera -
5000mAh battery - great for Instagram and all-day use.
...
CustomerAgent: PURCHASE CONFIRMED

RetailAdvisor: Congratulations! Your Redmi Note 13 order is confirmed.

```

Key Concepts Covered

Concept	Description
config_list / llm_config	Connects AutoGen to any OpenAI-compatible endpoint — here, local Ollama.
system_message	Defines an agent's persona, goals, and constraints (e.g., budget, what to ask).
initiate_chat()	Starts a two-agent conversation loop and returns a ChatResult with full history.
max_consecutive_auto_reply / max_turns	Safety limits that stop an agent-to-agent loop from running forever.

Lab 3: GroupChat Retail — Round Robin (4 Agents)

Source Code

```
# =====
# HANDS-ON 3 -- GroupChat Retail (ROUND ROBIN VERSION)
# =====

from autogen import ConversableAgent, GroupChat, GroupChatManager

print("\n=== HANDS-ON 3: GROUPCHAT RETAIL (ROUND ROBIN) ===\n")

# -- STEP 1: LLM CONFIGURATION -----

config_list = [
    {
        "model": "qwen2.5:3b",
        "base_url": "http://localhost:11434/v1",
        "api_key": "ollama",
        "price": [0, 0]
    }
]

llm_config = {
    "config_list": config_list,
    "temperature": 0.2
}

print("LLM Ready: qwen2.5:3b via Ollama")

# -- STEP 2: CUSTOMER AGENT -----

customer = ConversableAgent(
    name="Customer",
    system_message=(
        "You are a customer looking for a gaming laptop under Rs 70,000. "
        "You need 16GB RAM, RGB keyboard and a good GPU. "
        "When you are happy with the recommendation, say DEAL ACCEPTED."
    ),
    llm_config=llm_config,
    human_input_mode="NEVER"
)

# -- STEP 3: PRODUCT EXPERT -----

product_expert = ConversableAgent(
    name="ProductExpert",
    system_message=(
        "You are a gaming laptop specialist. "
        "Recommend ONE laptop from:\n"
        "1. Lenovo LOQ - Rs 68,000 - RTX 4050 - 16GB RAM\n"
        "2. ASUS TUF A15 - Rs 72,000 - RTX 4060 - 16GB RAM\n"
        "3. HP Victus 15 - Rs 58,000 - GTX 1650 - 8GB RAM\n\n"
        "Mention laptop name, GPU, RAM and price."
    ),
    llm_config=llm_config,
    human_input_mode="NEVER"
)

# -- STEP 4: BUDGET EXPERT -----

budget_expert = ConversableAgent(
    name="BudgetExpert",
    system_message=(
        "You are a pricing analyst. "
        "Check whether the recommendation fits within Rs 70,000. "
        "Respond with:\n"
    )
)
```

```

        "Within Budget\n"
        "Slightly Over Budget\n"
        "Over Budget\n"
        "Suggest discounts if needed."
    ),
    llm_config=llm_config,
    human_input_mode="NEVER"
)

# -- STEP 5: STORE MANAGER -----

store_manager = ConversableAgent(
    name="StoreManager",
    system_message=(
        "You are the store manager. "
        "Read all previous recommendations and provide ONE final decision. "
        "Mention final price and one reason to buy."
    ),
    llm_config=llm_config,
    human_input_mode="NEVER"
)

print("\nAgents Created:")
print("  Customer")
print("  ProductExpert")
print("  BudgetExpert")
print("  StoreManager")

# -- STEP 6: BUILD GROUPCHAT -----

print("\n--- Building GroupChat (ROUND ROBIN) ---")

group_chat = GroupChat(
    agents=[customer, product_expert, budget_expert, store_manager],
    messages=[],
    max_round=8,
    speaker_selection_method="round_robin"
)

manager = GroupChatManager(
    groupchat=group_chat,
    llm_config=llm_config
)

print(f"Agents in room : {[a.name for a in group_chat.agents]}")
print(f"Max rounds      : {group_chat.max_round}")
print(f"Speaker method    : {group_chat.speaker_selection_method}")

# -- STEP 7: START CONVERSATION -----

print("\n=== RETAIL GROUP CHAT ===\n")

result = customer.initiate_chat(
    manager,
    message=(
        "Hi team! I need a gaming laptop under Rs 70,000. "
        "I want a good GPU, 16GB RAM and RGB keyboard. "
        "Please suggest the best option."
    )
)

# -- STEP 8: ANALYSIS -----

print("\n\n=== CONVERSATION ANALYSIS ===")

messages = group_chat.messages

```

```

print(f"Total Messages : {len(messages)}")

print("\nSpeaker Breakdown:")
counts = {}
for msg in messages:
    speaker = msg.get("name", msg.get("role", "?"))
    counts[speaker] = counts.get(speaker, 0) + 1
for speaker, count in counts.items():
    print(f" {speaker:<20} ({count})")

# -- STEP 9: SPEAKING ORDER -----

print("\nSpeaking Order:")
for i, msg in enumerate(messages):
    speaker = msg.get("name", msg.get("role", "?"))
    print(f" Turn {i+1:2d}: {speaker}")

```

Sample Input / Output

```

"Hi team! I need a gaming laptop under Rs 70,000.
I want a good GPU, 16GB RAM and RGB keyboard."

--- Output ---
Agents in room : ['Customer', 'ProductExpert', 'BudgetExpert', 'StoreManager']
Max rounds      : 8
Speaker method  : round_robin

ProductExpert: I recommend the Lenovo LOQ - RTX 4050 - 16GB RAM - Rs 68,000.
BudgetExpert: Within Budget.
StoreManager: Final decision: Lenovo LOQ at Rs 68,000 - best GPU-to-price ratio.
Customer: DEAL ACCEPTED

```

Key Concepts Covered

Concept	Description
GroupChat	Holds multiple agents and a shared message list for multi-party conversations.
GroupChatManager	An LLM-backed orchestrator that decides whose turn it is to speak next.
speaker_selection_method='round_robin'	Cycles through agents in a fixed, predictable order.
max_round	Caps the total number of speaking turns in the group chat.

Lab 4: Capstone — Smart E-Commerce System (Tools + 6-Agent GroupChat + SQLite)

Source Code

```
# =====
# HANDS-ON 4 -- Smart E-Commerce Capstone (Qwen 2.5 via Ollama)
# =====

import sqlite3
import random
import datetime
import os
from autogen import ConversableAgent, GroupChat, GroupChatManager
from rich.console import Console
from rich.panel import Panel
from rich.table import Table

console = Console()
DB_PATH = os.path.join(os.path.dirname(__file__), "orders.db")

# -- SECTION 1: DATABASE SETUP -----

def setup_database():
    """Create the orders table if it does not exist."""
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS orders (
            id            INTEGER PRIMARY KEY AUTOINCREMENT,
            order_id      TEXT NOT NULL,
            customer_name TEXT NOT NULL,
            product       TEXT NOT NULL,
            original_price INTEGER,
            discount_pct  INTEGER,
            final_price   INTEGER,
            decision      TEXT,
            timestamp     TEXT
        )
    """)
    conn.commit()
    conn.close()

# -- SECTION 2: PRODUCT CATALOG -----

PRODUCT_CATALOG = {
    "Lenovo LOQ": {"price": 68000, "stock": 5, "rating": 4.6, "gpu": "RTX 4050", "ram": "16GB"},
    "ASUS TUF A15": {"price": 72000, "stock": 3, "rating": 4.5, "gpu": "RTX 4060", "ram": "16GB"},
    "HP Victus 15": {"price": 58000, "stock": 8, "rating": 4.3, "gpu": "GTX 1650", "ram": "8GB"},
    "Acer Nitro V": {"price": 65000, "stock": 0, "rating": 4.4, "gpu": "RTX 4050", "ram": "16GB"},
    "Dell G15": {"price": 79000, "stock": 2, "rating": 4.7, "gpu": "RTX 4060", "ram": "16GB"},
    "MSI Thin GF63": {"price": 55000, "stock": 6, "rating": 4.2, "gpu": "GTX 1650", "ram": "8GB"},
}

# -- SECTION 3: TOOL FUNCTIONS (pure Python -- no LLM) -----
# Rule: Use Python for computation. Use LLM for reasoning.
# These functions run BEFORE the GroupChat starts.

def find_best_product(need: str, budget: int) -> dict:
    """Find the best product based on user need and budget (exact -> keyword -> budget-fit)."""
    need_lower = need.lower().strip()
    available = {n: d for n, d in PRODUCT_CATALOG.items() if d["stock"] > 0}

    for name, details in available.items():
        if need_lower == name.lower():
            return {"product": name, **details}
```

```

for name, details in available.items():
    if need_lower in name.lower():
        return {"product": name, **details}

words = need_lower.split()
matches = []
for name, details in available.items():
    score = sum(10 for w in words if w in name.lower())
    if score > 0:
        score += details["rating"]
        if details["price"] <= budget:
            score += 5
        matches.append((score, name, details))
if matches:
    matches.sort(reverse=True)
    _, best_name, best_details = matches[0]
    return {"product": best_name, **best_details}

candidates = []
for name, details in available.items():
    score = details["rating"] * 10
    if details["price"] <= budget:
        score += 20
    score -= abs(details["price"] - budget) / 5000
    candidates.append((score, name, details))
candidates.sort(reverse=True)
_, best_name, best_details = candidates[0]
return {"product": best_name, **best_details}

def check_inventory(product_name: str) -> dict:
    """Check stock for a given product name."""
    for name, details in PRODUCT_CATALOG.items():
        if name.lower() in product_name.lower() or product_name.lower() in name.lower():
            return {"product": name, "available": details["stock"] > 0,
                    "stock_count": details["stock"], "price": details["price"],
                    "rating": details["rating"], "gpu": details["gpu"], "ram": details["ram"]}
    return {"product": product_name, "available": False, "stock_count": 0,
            "price": 0, "rating": 0, "gpu": "Unknown", "ram": "Unknown"}

def calculate_discount(price: int, budget: int) -> dict:
    """Within budget -> 5%; up to 10% over -> 8%; up to 15% over -> 10%; beyond -> 0%."""
    if price <= budget:
        pct = 5
    elif price <= budget * 1.10:
        pct = 8
    elif price <= budget * 1.15:
        pct = 10
    else:
        pct = 0
    final = int(price * (1 - pct / 100))
    return {"original_price": price, "discount_pct": pct, "final_price": final,
            "within_budget": final <= budget}

def generate_order_id() -> str:
    """Generate unique order ID: ORD + 4 random digits."""
    return "ORD" + str(random.randint(1000, 9999))

def save_order(order_id, customer_name, product, original_price,
               discount_pct, final_price, decision) -> bool:
    """Save the completed order to SQLite. Returns True on success."""
    try:
        conn = sqlite3.connect(DB_PATH)
        cursor = conn.cursor()
        cursor.execute(

```

```

        "INSERT INTO orders (order_id, customer_name, product, original_price, "
        "discount_pct, final_price, decision, timestamp) VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
        (order_id, customer_name, product, original_price, discount_pct,
         final_price, decision, datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S"))
    )
    conn.commit()
    conn.close()
    return True
except Exception as e:
    console.print(f"[red]DB error: {e}[/red]")
    return False

def generate_invoice(order_id, customer_name, product, original_price,
                    discount_pct, final_price, decision) -> str:
    """Generate a formatted receipt string."""
    sep = "=" * 50
    row = [sep, "          SMART E-COMMERCE -- ORDER RECEIPT", sep,
           "Order ID       : " + str(order_id),
           "Date           : " + datetime.datetime.now().strftime("%d %b %Y  %H:%M"), sep,
           "Customer        : " + str(customer_name),
           "Product         : " + str(product),
           "Original Price : Rs " + "{:,}".format(original_price),
           "Discount         : " + str(discount_pct) + "%",
           "Final Price      : Rs " + "{:,}".format(final_price), sep,
           "Decision          : " + str(decision), sep,
           "Thank you for shopping with us!", sep]
    return "\n" + "\n".join(row) + "\n"

# -- SECTION 4: LLM CONFIG -----

config_list = [
    {"model": "qwen2.5:3b", "base_url": "http://localhost:11434/v1", "api_key": "ollama"}
]
llm_config = {"config_list": config_list, "temperature": 0.2}

# -- SECTION 5: MAIN APPLICATION -----

def run_ecommerce_system():
    setup_database()
    console.print("\n[n[bold magenta]=== SMART E-COMMERCE SYSTEM ===[/bold magenta]\n")

    customer_name = input("Customer Name : ").strip() or "Shyni"
    budget_str = input("Budget (Rs) : ").strip()
    budget = int(budget_str) if budget_str.isdigit() else 70000
    need = input("What do you need? (e.g. gaming laptop): ").strip() or "gaming laptop"

    console.print("\n[n[cyan]Customer : " + customer_name + "[/cyan]")
    console.print("[cyan]Budget : Rs " + "{:,}".format(budget) + "[/cyan]")
    console.print("[cyan]Need : " + need + "[/cyan]\n")
    console.print("[bold yellow]Running tools...[/bold yellow]")

    # -- Deterministic tool calls (no LLM) run first --
    product_data = find_best_product(need, budget)
    inv_data = check_inventory(product_data["product"])
    disc_data = calculate_discount(inv_data["price"], budget)
    order_id = generate_order_id()

    table = Table(title="Tool Results", style="bold")
    table.add_column("Field", style="cyan", width=20)
    table.add_column("Value", style="white", width=30)
    table.add_row("Product", inv_data["product"])
    table.add_row("GPU", inv_data["gpu"])
    table.add_row("RAM", inv_data["ram"])
    table.add_row("Rating", str(inv_data["rating"]) + " / 5.0")
    stock_str = str(inv_data["stock_count"]) + " units " + ("IN STOCK" if inv_data["available"] else
"OUT OF STOCK")
    table.add_row("Stock", stock_str)

```

```

table.add_row("Original Price", "Rs " + "{:,}".format(inv_data["price"]))
table.add_row("Discount", str(disc_data["discount_pct"]) + "%")
table.add_row("Final Price", "Rs " + "{:,}".format(disc_data["final_price"]))
table.add_row("Within Budget", "Yes" if disc_data["within_budget"] else "No")
table.add_row("Order ID", order_id)
console.print(table)

status_str = "IN STOCK" if inv_data["available"] else "OUT OF STOCK"

# -- Six specialist agents, each grounded in tool output --
customer_agent = ConversableAgent(name="Customer", llm_config=llm_config,
human_input_mode="NEVER",
    system_message="You are " + customer_name + ", looking for a " + need + ". Budget Rs "
    + "{:,}".format(budget) + ". State your requirement. When StoreManager gives the final "
    "recommendation, say BUY if satisfied or PASS if not.")

product_agent = ConversableAgent(name="ProductAgent", llm_config=llm_config,
human_input_mode="NEVER",
    system_message="You are the Product Specialist. Recommended: " + inv_data["product"] +
    ". GPU: " + inv_data["gpu"] + ", RAM: " + inv_data["ram"] + ", Rating: " +
str(inv_data["rating"]) +
    "/5.0. Price: Rs " + "{:,}".format(inv_data["price"]) + ". Explain in 2-3 sentences why
this matches.")

inventory_agent = ConversableAgent(name="InventoryAgent", llm_config=llm_config,
human_input_mode="NEVER",
    system_message="You are the Inventory Manager. " + inv_data["product"] + " is " + status_str +
    " with " + str(inv_data["stock_count"]) + " units. Report availability; mention urgency if
under 5 units.")

pricing_agent = ConversableAgent(name="PricingAgent", llm_config=llm_config,
human_input_mode="NEVER",
    system_message="You are the Pricing Analyst. Original: Rs " + "{:,}".format(inv_data["price"])
+
    ". Budget: Rs " + "{:,}".format(budget) + ". Discount: " + str(disc_data["discount_pct"])
+
    "%. Final: Rs " + "{:,}".format(disc_data["final_price"]) + ". State clearly in 2
sentences.")

review_agent = ConversableAgent(name="ReviewAgent", llm_config=llm_config,
human_input_mode="NEVER",
    system_message="You are the Review Analyst. " + inv_data["product"] + " rating " +
    str(inv_data["rating"]) + "/5.0. Classify Excellent/Good/Average/Poor with one sentence.")

manager_agent = ConversableAgent(name="StoreManager", llm_config=llm_config,
human_input_mode="NEVER",
    system_message="You are the Store Manager. After hearing all agents, give ONE final
recommendation "
    "to " + customer_name + " with product, final price Rs " +
"{:,}".format(disc_data["final_price"]) +
    ", and order ID " + order_id + ". End with exactly one word: BUY or NEGOTIATE or PASS")

# -- GroupChat with automatic (LLM-driven) speaker selection --
group_chat = GroupChat(
    agents=[customer_agent, product_agent, inventory_agent, pricing_agent, review_agent,
manager_agent],
    messages=[], max_round=10, speaker_selection_method="auto"
)
gc_manager = GroupChatManager(groupchat=group_chat, llm_config=llm_config)

console.print("\n[bold cyan]Starting 6-agent GroupChat...[/bold cyan]\n")

customer_agent.initiate_chat(gc_manager, message=(
    "Hi everyone! I am " + customer_name + ". I am looking for a " + need + " with budget Rs "
    + "{:,}".format(budget) + ". Please help me make the best decision.))

# -- Parse StoreManager's final word --

```

```

final_decision = "BUY"
for msg in reversed(group_chat.messages):
    if msg.get("name") == "StoreManager":
        content = msg.get("content", "").upper()
        final_decision = "PASS" if "PASS" in content else "NEGOTIATE" if "NEGOTIATE" in content
else "BUY"
    break

# -- Deterministic persistence + invoice (no LLM) --
save_order(order_id, customer_name, inv_data["product"], inv_data["price"],
           disc_data["discount_pct"], disc_data["final_price"], final_decision)
invoice = generate_invoice(order_id, customer_name, inv_data["product"], inv_data["price"],
                          disc_data["discount_pct"], disc_data["final_price"], final_decision)

console.print(Panel(invoice, title="[bold green]ORDER RECEIPT[/bold green]",
border_style="green"))
console.print("[dim]Saved to: " + DB_PATH + "[/dim]")

if __name__ == "__main__":
    run_ecommerce_system()

```

Sample Input / Output

```

Customer Name : Shyni
Budget (Rs)   : 70000
What do you need? gaming laptop

--- Output ---
Tool Results
Product: Lenovo LOQ | GPU: RTX 4050 | RAM: 16GB | Stock: 5 units IN STOCK
Discount: 5% | Final Price: Rs 64,600 | Order ID: ORD4821

ORDER RECEIPT
=====
Customer: Shyni | Product: Lenovo LOQ
Final Price: Rs 64,600 | Decision: BUY
=====
Saved to: orders.db

```

Key Concepts Covered

Concept	Description
Deterministic Tool Functions	find_best_product/check_inventory/calculate_discount run in plain Python before any LLM call.
speaker_selection_method='auto'	The GroupChatManager's LLM dynamically decides the next speaker based on context.
AI + Business Logic Hybrid	Agents reason over tool outputs; SQLite persistence and invoice text are pure Python.
Grounded System Messages	Each agent's system_message is dynamically built from tool results (price, stock, rating).

Lab 5: Human-in-the-Loop Loan Approval (Interactive Console)

Source Code

```
# =====
# HANDS-ON 5 -- Human-in-the-Loop Loan Approval System
# =====

from autogen import ConversableAgent
from rich.console import Console
from rich.panel import Panel
from rich.table import Table

console = Console()

# -- LLM CONFIG -----

config_list = [
    {"model": "qwen2.5:3b", "base_url": "http://localhost:11434/v1", "api_key": "ollama", "price": [0,
0]}
]
llm_config = {"config_list": config_list, "temperature": 0.2}

console.print(Panel.fit("[bold cyan]AI LOAN APPROVAL SYSTEM[/bold cyan]", border_style="green"))

# -- CUSTOMER APPLICATION (console input) -----

customer_name = input("Customer Name      : ")
loan_amount = input("Loan Amount (Rs)    : ")
monthly_income = input("Monthly Income (Rs): ")

console.print()
table = Table(title="Loan Application")
table.add_column("Field", style="cyan")
table.add_column("Value", style="green")
table.add_row("Customer", customer_name)
table.add_row("Loan Amount", f"Rs {loan_amount}")
table.add_row("Monthly Income", f"Rs {monthly_income}")
console.print(table)

# -- RISK AGENT -----

risk_agent = ConversableAgent(
    name="RiskAgent",
    system_message="""
You are a banking risk analyst.
Analyze the loan request.
Return:
Risk Level: Low / Medium / High
Keep answer short.
""",
    llm_config=llm_config,
    human_input_mode="NEVER"
)

# -- CREDIT SCORE AGENT -----

credit_agent = ConversableAgent(
    name="CreditScoreAgent",
    system_message="""
You are a credit score specialist.
Generate a realistic credit score between 650 and 850.
Format:
Credit Score: <score>
Recommendation:
APPROVE
REVIEW
"""
)
```

```

    REJECT
    Keep answer short.
    """
    llm_config=llm_config,
    human_input_mode="NEVER"
)

# -- HUMAN BANK MANAGER (human_input_mode="ALWAYS") -----

bank_manager = ConversableAgent(
    name="BankManager",
    llm_config=False,
    human_input_mode="ALWAYS"
)

# -- RISK ASSESSMENT (generate_reply -- single LLM call, no back-and-forth) -----

console.print(Panel.fit("[bold yellow]Running Risk Assessment...[/bold yellow]",
border_style="yellow"))

risk_result = risk_agent.generate_reply(
    messages=[{"role": "user", "content":
        f"Customer: {customer_name}\nLoan Amount: Rs {loan_amount}\nMonthly Income: Rs {monthly_income}"}]
)
console.print(Panel(risk_result, title="Risk Agent", border_style="red"))

# -- CREDIT SCORE -----

credit_result = credit_agent.generate_reply(
    messages=[{"role": "user", "content":
        f"Customer: {customer_name}\nLoan Amount: Rs {loan_amount}\nMonthly Income: Rs {monthly_income}"
        f"\nRisk Analysis:\n{risk_result}"}]
)
console.print(Panel(credit_result, title="Credit Score Agent", border_style="blue"))

# -- HUMAN-IN-THE-LOOP: a real person types the final decision -----

console.print()
console.print(Panel.fit("[bold magenta]HUMAN APPROVAL REQUIRED[/bold magenta]",
border_style="magenta"))
console.print(f"\n[bold]Risk Agent:[/bold]\n{risk_result}\n\n[bold]Credit Agent:[/bold]\n{credit_result}\n")

decision = input("\nApprove loan of Rs " + loan_amount + "? (yes/no): ").strip().lower()

# -- FINAL DECISION -----

if decision == "yes":
    console.print(Panel.fit("[bold green]LOAN APPROVED[/bold green]", border_style="green"))
else:
    console.print(Panel.fit("[bold red]LOAN REJECTED[/bold red]", border_style="red"))

console.print(Panel.fit("[bold cyan]Workflow Completed[/bold cyan]", border_style="cyan"))

```

Sample Input / Output

```

Customer Name      : Ravi
Loan Amount (Rs)   : 500000
Monthly Income (Rs): 60000
Approve loan of Rs 500000? (yes/no): yes

```

```

--- Output ---

```

Risk Agent: Risk Level: Medium
 Credit Score Agent: Credit Score: 712
 Recommendation: REVIEW

HUMAN APPROVAL REQUIRED
 LOAN APPROVED
 Workflow Completed

Key Concepts Covered

Concept	Description
<code>generate_reply()</code>	Gets a single LLM response for a message list, without a full back-and-forth chat loop.
<code>human_input_mode='ALWAYS'</code>	Forces this agent (BankManager) to always prompt a real person for input.
Human-in-the-Loop Pattern	AI agents (Risk, Credit) advise; a human makes the final, authoritative decision.
Rich Console UI	Panels and Tables present agent outputs and the application form clearly.

Lab 6: Loan Approval with `initiate_chat()` Human Hand-off

Source Code

```
# =====
# HANDS-ON 6 -- Loan Approval with initiate_chat() Human Hand-off
# =====

from autogen import ConversableAgent

# -- LLM CONFIG -----

config_list = [
    {"model": "qwen2.5:3b", "base_url": "http://localhost:11434/v1", "api_key": "ollama", "price": [0,
0]}
]
llm_config = {"config_list": config_list, "temperature": 0.2}

# -- RISK AGENT -----

risk_agent = ConversableAgent(
    name="RiskAgent",
    system_message="""
You are a banking risk analyst.
Return:
Risk Level: Low / Medium / High
""",
    llm_config=llm_config,
    human_input_mode="NEVER"
)

# -- CREDIT AGENT -----

credit_agent = ConversableAgent(
    name="CreditAgent",
    system_message="""
Generate a credit score and recommendation.
Format:
Credit Score: <score>
Recommendation:
APPROVE / REVIEW / REJECT
""",
    llm_config=llm_config,
    human_input_mode="NEVER"
)

# -- HUMAN BANK MANAGER -----

bank_manager = ConversableAgent(
    name="BankManager",
    llm_config=False,
    human_input_mode="ALWAYS"
)

# -- CUSTOMER DATA (static, for repeatable demo) -----

customer_name = "Ravi"
loan_amount = "500000"
monthly_income = "60000"

# -- RISK ANALYSIS -----

risk_result = risk_agent.generate_reply(
    messages=[{"role": "user", "content":
f"Customer: {customer_name}\nLoan Amount: {loan_amount}\nMonthly Income: {monthly_income}"}]
)
```

```

print("\n=== Risk Analysis ===")
print(risk_result)

# -- CREDIT ANALYSIS -----

credit_result = credit_agent.generate_reply(
    messages=[{"role": "user", "content":
        f"Customer: {customer_name}\nLoan Amount: {loan_amount}\nMonthly Income: {monthly_income}"
        f"\nRisk Analysis:\n{risk_result}"}]
)
print("\n=== Credit Analysis ===")
print(credit_result)

# -- HUMAN-IN-THE-LOOP via initiate_chat() (credit_agent hands off to the human) -----

print("\n=== HUMAN APPROVAL REQUIRED ===\n")

credit_agent.initiate_chat(
    bank_manager,
    message=f"""
Loan Application Summary

Customer: {customer_name}
Loan Amount: Rs {loan_amount}
Monthly Income: Rs {monthly_income}

{risk_result}

{credit_result}

Should this loan be approved?
Type APPROVE or REJECT.
""",
    max_turns=1
)

```

Sample Input / Output

Customer: Ravi | Loan Amount: 500000 | Monthly Income: 60000

--- Output ---

=== Risk Analysis ===
Risk Level: Medium

=== Credit Analysis ===
Credit Score: 705
Recommendation: REVIEW

=== HUMAN APPROVAL REQUIRED ===
BankManager (human): APPROVE

Key Concepts Covered

Concept	Description
initiate_chat() to a Human Agent	credit_agent hands off directly to bank_manager (human_input_mode='ALWAYS').
max_turns=1	Limits the hand-off chat to exactly one human response, avoiding further back-and-forth.

Concept	Description
Static Demo Data	Hard-coded customer data makes the human-in-the-loop hand-off repeatable for testing.

Cheat Sheet

Syntax / Call	Purpose
<code>ConversableAgent(name, llm_config, human_input_mode)</code>	Create an agent, with or without an LLM.
<code>llm_config=False</code>	Disable the LLM — agent only relays/echoes messages you send it.
<code>agent.send(message, recipient, request_reply=False)</code>	Directly deliver a message without invoking an LLM reply.
<code>agent.chat_messages.get(partner, [])</code>	Read the message history between two specific agents.
<code>agent_a.initiate_chat(agent_b, message, max_turns=N)</code>	Run a full multi-turn conversation between two agents.
<code>agent.generate_reply(messages=[...])</code>	Get one LLM response for a custom message list (no chat loop).
<code>GroupChat(agents, messages, max_round, speaker_selection_method)</code>	Define a multi-party conversation room.
<code>GroupChatManager(groupchat, llm_config)</code>	LLM-driven orchestrator that manages turn-taking in a GroupChat.
<code>speaker_selection_method='round_robin' 'auto'</code>	Fixed order vs. LLM-decided next speaker.
<code>human_input_mode='NEVER' 'ALWAYS' 'TERMINATE'</code>	Controls whether/when a real person is prompted.
<code>max_consecutive_auto_reply=N</code>	Caps how many times one agent can auto-reply in a row.

Industry Use Cases

Pattern	Real-World Application
No-LLM message passing	Simulating organisational message flows, workflow/event-bus prototyping.
Two-agent LLM chat	Sales/retail chatbots, negotiation bots, tutoring or Q&A assistants.
Round-robin GroupChat	Structured panel discussions, multi-perspective review pipelines (e.g., budget + product + manager sign-off).
Auto-speaker GroupChat + tools	E-commerce recommendation engines, dynamic multi-specialist customer support.
Human-in-the-loop	Loan/credit approval, medical triage escalation, content moderation requiring sign-off.

Troubleshooting

Issue	Solution
Connection refused to localhost:11434	Ensure Ollama is running ('ollama serve') and the model is pulled ('ollama pull qwen2.5:3b').
Agents reply forever / conversation never ends	Set/lower max_turns, max_round, or max_consecutive_auto_reply.
GroupChatManager picks the same speaker repeatedly	Switch speaker_selection_method to 'round_robin' for predictable rotation, or refine system_messages.
Agent gives empty or generic replies	Make system_message more specific; lower temperature for consistency.
human_input_mode='ALWAYS' agent doesn't prompt	Confirm the agent is actually reached in the chat flow (correct recipient/manager wiring).
sqlite3 errors in the capstone (Lab 4)	Delete orders.db and let setup_database() recreate the table.

Conclusion

Across these six labs, students progressed from raw message-passing between agents with no LLM, to two-agent LLM-driven negotiations, to multi-agent GroupChats with both fixed and AI-driven speaker selection, and finally to human-in-the-loop approval workflows. AutoGen's conversation-centric design makes it straightforward to model real-world multi-party interactions — and, as the capstone shows, to combine that with deterministic Python tools and persistence for production-style hybrid systems.